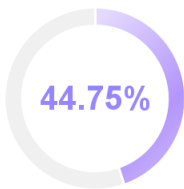


NO. 6d6dbc701fkt9df4 | 2026-05-24 21:25:21

- 题目：基于 ECS 架构的类吸血鬼幸存者游戏开发及优化（校外）
- 作者：刘瀚文
- 检测所属单位： -

📄 论文字符数：34046 📄 论文页数：55 📊 表格数量：22 🖼️ 图片数量：13

检测结果



44.75%
疑似AIGC生成占比

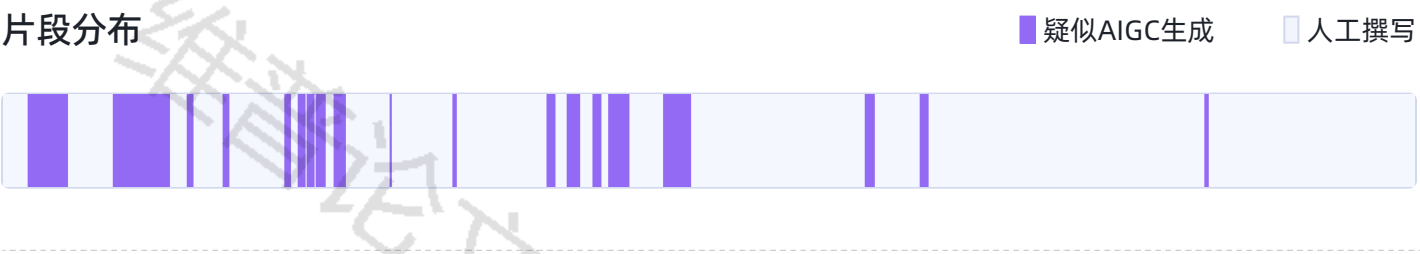
55.25%
人工撰写占比

结果分布

序号	章节	疑似AIGC生成文字/章节总字数	疑似AIGC生成章节占比	人工撰写占比
1	摘要	413/413	100.0%	0.0%
2	Abstract	167/168	99.0%	1.0%
3	第一章 绪论	2328/2766	84.0%	16.0%
4	第二章 相关技术与理论基础	2165/4867	44.0%	56.0%
5	第三章 系统需求分析	214/2723	7.0%	93.0%
6	第四章 系统总体设计	2785/3848	72.0%	28.0%
7	第五章 核心模块详细设计与实现	667/2681	24.0%	76.0%

8	第六章 性能优化与工程实践	0/1498	0.0%	100.0%
9	第七章 系统测试与验证	350/2870	12.0%	88.0%
10	第八章 总结与展望	146/915	15.0%	85.0%

*注:格式规范的情况下可准确识别章节, 若论文中无章节, 可能会识别有误。



基于 ECS 架构的类吸血鬼幸存者游戏开发及优化（校外）

中国石油大学 (华东)

CHINA UNIVERSITY OF PETROLEUM

本科毕业设计（论文）

题目：基于 ECS 架构的类吸血鬼幸存者游戏开发及优化（校外）

学生姓名：刘瀚文
学号：2207020509
专业班级：计算机科学与技术专业
指导教师：董玉坤

2026年5月24日

学位论文原创性声明

本人所提交的学位论文《基于 ECS 架构的类吸血鬼幸存者游戏开发及优化（校外）》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

论文作者（签名）： 指导教师确认（签名）：

年 月 日 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

保密的学位论文在_____年解密后适用本授权书。

论文作者（签名）： 指导教师（签名）：

年 月 日 年 月 日

摘 要

随着 HTML5 与 WebGL 技术的普及，浏览器端实时动作游戏在架构解耦与同屏性能方面面临新的挑战。类吸血鬼幸存者玩法融合 Roguelite 单局构筑与元游戏流程，要求客户端在数据驱动、模块划分与可验证性上具备清晰边界。本文以 Survivor And Fight 毕业设计为研究对象，基于 LayaAir 3.x 与 TypeScript，对 2D 俯视角生存战斗游戏开展从需求分析、总体设计、系统实现到测试验证的完整研究。

论文主体共分八章：绪论与相关技术奠定选题依据与技术路线；需求分析明确功能、非功能需求及验收映射；总体设计提出五层架构，并从 ECS Gameplay、MVC 界面、对象池与 Web Worker 三方面给出方案；实现与性能章节分别论述核心模块、主循环及优化实践；测试章节通过单元脚本、功能用例与压力实验进行验证；结论章节归纳成果与不足。原型可稳定支撑菜单、跑图、战斗与技能装配等闭环流程，对象池与动态图集可改善高同屏负载下的运行表现。

关键词：实体组件系统；类吸血鬼幸存者；系统设计与实现；LayaAir；数据驱动

Abstract

With the adoption of HTML5 and WebGL, browser-based action games must balance architectural decoupling and runtime performance under large entity counts. Vampire-survivor-like gameplay combined with Roguelite meta progression calls for data-driven content, modular organization, and verifiable engineering practices. This thesis studies the off-campus project Survivor And Fight: a 2D top-down survival combat prototype on LayaAir 3.x and TypeScript, covering requirements analysis, overall design, implementation, and testing.

The thesis is organized in eight chapters. The introduction and related work establish motivation and technology choices; requirements analysis defines functional and non-functional criteria; overall design presents a layered architecture with ECS gameplay, MVC-based UI, object pooling, and Web Worker offloading; implementation and performance chapters describe core modules, the main loop, and optimization practice; testing covers unit scripts, functional cases, and stress experiments; the conclusion summarizes contributions and limitations. The prototype supports a

full loop of menus, run-map navigation, combat, and skill loadout. Experiments show that pooling and dynamic atlasing improve stability under high on-screen load.

Keywords: ECS; vampire-survivor-like; system design; LayaAir; data-driven

目 录

第一章 绪论1

1.1研究背景与意义1

1.2国内外研究现状1

1.2.1 游戏引擎与 H5 开发平台研究现状1

1.2.2 实体组件系统与玩法架构2

1.3 主要创新点与特色3

1.4 伦理与合规说明3

1.5 相关游戏与竞品简要分析3

1.6 术语与缩写预备说明4

1.7 本章小结4

第二章 相关技术与理论基础5

2.1 引擎语言选择5

2.1.1 H5游戏引擎选型5

2.1.2 TypeScript与工程常量管理6

2.1.3 主循环、预制体与引擎使用约束6

2.2 项目玩法架构选择7

2.2.1 Survivor-like品类与核心循环7

2.2.2 轻量实体组件系统（ECS）7

2.2.3 数据驱动与JSON配表8

2.2.4 Roguelite跑图与关卡结构8

2.2.5 战斗性能与密集实体仿真9

2.3 UI架构选择9

2.3.1 MVC分层与全屏界面栈9

2.3.2 战斗内UI：HUD、暂停与技能装配10

2.3.3 UI架构选型的综合评价10

2.4 本章小结10

第三章 系统需求分析11

3.1 可行性分析11

3.1.1 技术可行性11

3.1.2 经济与社会可行性11

3.2 用户角色与运行环境	11
3.2.1 用户角色	11
3.2.2 运行环境	12
3.3 功能性需求	12
3.3.1 用例：局内战斗	12
3.3.2 用例：元游戏进入战斗	12
3.4 功能需求详细说明	13
3.4.1 ECS核心	13
3.4.2 战斗与技能	13
3.4.3 怪物、元游戏与UI	13
3.5 非功能性需求	13
3.6 数据、接口与约束	14
3.7 需求优先级	14
3.8 需求与测试映射	14
3.9 本章小结	14
第四章 系统总体设计	15
4.1 设计原则	15
4.2 系统具体设计	15
4.2.1 基于ECS的Gameplay实现	15
4.2.2 基于MVC的UI结构实现	16
4.2.3 基于Web Worker和对象池的优化实现	17
4.3 系统交互与时序	17
4.4 可靠性设计	18
4.5 容错与降级策略	19
4.6 可扩展性设计	20
4.7 本章小结	20
第五章 核心模块详细设计与实现	21
5.1 ECS架构GamePlay实现	21
5.1.1 世界、组件与筛选器	21
5.1.2 属性、移动与自动施法	21
5.1.3 技能与效果执行链	21
5.1.4 子弹碰撞与怪物AI	22
5.1.5 经验、升级与元游戏数据	22
5.2 MVC架构UI实现	22

- 5.2.1 UI栈与元游戏界面22
- 5.2.2 战斗HUD与技能装配22
- 5.2.3 输入与重启面板23
- 5.3 Web Worker和对象池的优化实现23
- 5.4 游戏主循环设计24
 - 5.4.1 入口与配置时序24
 - 5.4.2 SimpleEcsDemo构造与init24
 - 5.4.3 帧循环与暂停24
- 5.5 典型场景实现剖析24
 - 5.5.1 场景一：玩家首次进入战斗24
 - 5.5.2 场景二：Tab打开技能面板并更换装备24
 - 5.5.3 场景三：怪物密集时的Worker卸载25
- 5.6 关键代码文件索引25
- 5.7 功能模块与论文章节对照25
- 5.8 本章小结26

第六章 性能优化与工程实践27

- 6.1 性能目标与瓶颈27
- 6.2 对象池27
- 6.3 Web Worker卸载27
- 6.4 渲染与逻辑侧其它优化28
- 6.5 帧预算与内存28
- 6.6 性能实验设计28
 - 6.6.1 实验目的与变量28
 - 6.6.2 实验步骤28
 - 6.6.3 优化自检清单29
- 6.7 绿色计算与兼容性29
- 6.8 本章小结29

第七章 系统测试与验证30

- 7.1 测试策略30
- 7.2 单元验证30
 - 7.2.1 ECS核心30
 - 7.2.2 公式解析器30
 - 7.2.3 属性修饰器30
- 7.3 功能测试30

7.3.1 集成测试补充	31
7.4 性能测试	31
7.4.1 测试环境	31
7.4.2 动态图集对比（前三波）	32
7.4.3 五波完整帧率	32
7.4.4 对象池消融（第4-5波）	32
7.5 系统运行截图	33
7.6 测试结论	33
7.7 本章小结	36
第八章 总结与展望	37
8.1 工作总结	37
8.2 不足与展望	37
8.3 社会、健康与文化影响	37
8.4 绿色节能与兼容性	38
8.5 本章小结	38
致 谢	39
参考文献	40
附 录	42
附录A 名词术语及缩略词	43
附录B 核心配表字段示例（节选）	44
附录C 需求与测试用例映射（节选）	45
附录D 缺陷记录（项目实测）	46

第一章 绪论

1.1 研究背景与意义

随着移动互联网与 Web 技术的普及，H5 游戏在独立作品开发与教学实践中应用日益广泛。相较原生安装包，H5 游戏具有链接即玩、跨端发布、逻辑与资源共用 Web 技术栈等优势，有利于降低分发与迭代成本。然而，浏览器端 JavaScript 运行时通常仅有一条主线程，垃圾回收（GC）与渲染绘制共享帧时间预算；在战斗类场景中，若单帧内集中处理大量实体移动、碰撞检测与界面刷新，玩家将直接感知到帧率下降，影响可玩性。

以《Vampire Survivors》（吸血鬼幸存者）为代表的类幸存者（Survivor-like）玩法，强调自动攻击、怪潮压迫与局内构筑，并常与 Roguelite 元进度、随机路线相结合。该类游戏对软件架构提出明确要求：玩法数据需支持外置配置与快速迭代，同屏需承载大量怪物与弹幕实体，主菜单、选关、跑图与局内战斗等流程需形成可闭环的元游戏链路。传统以面向对象、深层继承为主的游戏架构，在实体规模持续扩展时容易加剧模块耦合与架构退化，进而抬高维护成本并制约运行性能[1]。

实体组件系统 (Entity-Component-System, ECS) 通过将实体、数据组件与逻辑系统分离, 以组合替代继承, 有利于改善模块边界与批处理更新效率。LayaAir 等国产 H5 引擎在 2D 渲染、TypeScript 支持与多端发布方面较为成熟, 为“引擎能力 + 玩法架构 + 性能优化”的一体化毕设实践提供了可行平台。

基于上述背景, 本文以浏览器端 2D 俯视角 Roguelite 原型 Survivor And Fight 为研究对象, 基于 LayaAir 3.x 与 TypeScript, 开展从需求分析、总体设计、系统实现到测试验证的完整研究, 重点探索轻量 ECS、配表驱动技能、对象池与 Web Worker 等机制在 H5 生存战斗场景中的工程落地路径。

1.2 国内外研究现状

1.2.1 游戏引擎与 H5 开发平台研究现状

Gregory 在其专著中从工业实践出发, 对渲染、资源、场景图、动画与脚本等子系统进行了系统分层论述, 为理解商业引擎的整体结构提供了常用参照框架[2]。Munro 等对 Quake 系列引擎的架构拆解, 有助于区分引擎层与游戏层的职责边界[3]。Agrahari 等对 Unreal Engine 内部模块组织方式的分析, 则展示了现代大型引擎在功能划分上的典型形态[4]。相较之下, Goslin 等对 Panda3D 的介绍代表了脚本层叠于 C++ 核心之上的早期引擎实现路径[5]。

近年来, 研究重点进一步转向“架构变更的影响范围分析”。Ullmann 等提出的 SyDRA 方法可从开源引擎中恢复子系统依赖图, 并通过对照实验表明结构化可视化能够提升开发者对耦合关系的理解效率[6]。其后续工作进一步将耦合模式显式化, 指出目录嵌套与过度耦合是架构退化的重要来源[7]。

在 H5 场景下, 除通用引擎架构问题外, 还需额外考虑主线程时间预算、异步资源加载与运行环境沙箱限制。Unity 官方文档对脚本生命周期、渲染管线与平台差异的系统说明, 可作为理解浏览器端游戏运行时约束的公开技术参考[8]。现有引擎教程与示例多侧重功能演示, 对“可扩展玩法框架 + 可复现实验数据”的完整交付支持仍显不足。

1.2.2 实体组件系统与玩法架构

ECS 将游戏对象抽象为实体标识、纯数据组件与无状态系统, 通过按组件类型批处理更新, 改善逻辑耦合与数据局部性。Unity DOTS 代表了数据导向与并行化方向; 在毕设与中小规模项目中, 基于 Map 存储组件、单线程 tick 的轻量 ECS 因学习与排错成本较低而更易落地。

在玩法层面, Survivor-like 作品以波次管理与自动攻击为核心; Roguelite 作品则常通过 DAG 节点地图强化路线选择; 部分作品进一步通过模块化技能组合拓展构筑深度。相较上述商业产品, H5 生存类原型若仍采用预制体叠加面向对象结构, 往往在实体规模上升后面临维护与性能双重压力。Briand 等通过控制实验表明, 面向对象设计结构的选择会显著影响后续修改成本[9], 这也为采用 ECS 等组合式架构提供了工程层面的旁证。

在性能相关研究方面, 成百上千代理同时运动是生存战斗的典型热点。Reynolds 提出的 Boids 模型为局部规则驱动的群体运动提供了经典思路[10]。Thalman 等对人群仿真的系统论述则概括了由个体行为到群体涌现的常见建模路径[11]。Cantrell 等在 Unity 中构建物理驱动疏散模型, 并给出了完整的验证与确认 (V&V) 流程[12]。Bott 等在 Unreal Development Kit 上实现了基于物理工具链的人群运动方案[13]。Helbing 的恐慌模型进一步表明, 分离力对密集场景下的运动稳定性具有关键影响[14]。McKenzie 等将人群行为建模与游戏技术相结合, 为在实时仿真中控制计算开销提供了工程化参考[15]。需要指出的是, Web Worker 无法直接访问 DOM 与大部分引擎

API，因而不适合承担碰撞检测与渲染任务，仅适合纯数值计算的下放。

1.3 主要创新点与特色

本项目工程侧特点如下：

(1) 轻量 ECS：EcsWorld 管玩法 tick，ViewComponent 只绑 Laya 节点，业务少直接调引擎 API。

(2) 配表技能链：Skill 配多行 SkillEffect，修饰器排在子弹前，支持穿透、分裂、连锁；配表现有 70 余条效果行。

(3) 元流程联动：三幕 DAG 经 MetaFlowController 进战斗，Boss 节点可带更长生存时间等载荷。

(4) 性能措施：BulletPool/MonsterPool 减分配；达标时用 Worker 算追逐分离，碰撞仍主线程；池化改善 P95，图集抬高千怪波 FPS。

(5) UI 与战斗协同：UIStackManager 管全屏路由；Tab 装配时 GameSession.paused 冻逻辑，SkillLoadoutSyncSystem 写回施法状态。

以上条目可在原型和第 7 章数据里核对，并对应第 4 章、第 5 章。

1.4 伦理与合规说明

内容为虚构战斗，无真实暴力。不采集个人敏感信息。若对外发布，需遵守网游与未成年人相关规定，并提示适龄与时长[18]。社会影响在第 8 章另有说明。

1.5 相关游戏与竞品简要分析

吸血鬼幸存者：极简操作加自动攻击和怪海，证明品类成立[7]。本项目借其节奏和升级感，但用 ECS 和 JSON 表把逻辑摊开，方便写进论文。

杀戮尖塔：节点地图加路线风险[7]。本文跑图也用 DAG，战斗却是实时俯视，瓶颈在帧率和碰撞。

Noita：模块拼法术带来深度[7]。本文 effect 链只吸收“组合”思路，不做像素物理。

其他 H5 生存作：不少仍用预制体加 OOP。本文用 ECS、配表和 Worker，强调可维护和可测，不拼内容量。

1.6 术语与缩写预备说明

英文缩写首次出现给中文释义，见附录 A。类名与 API 保留英文，便于对照源码。

1.7 本章小结

本章交代背景、相关研究和本项目的工程特点，供后文需求与设计引用。

第二章 相关技术与理论基础

本章从引擎与语言、玩法与逻辑架构、界面架构三条线说明 Survivor And Fight 的技术选型及依据，为第 3 章需求与第 4 章设计提供共同语境。各节在文献与业界实践基础上给出毕业设计规模下的取舍，并与后文 SimpleEcsDemo、UIStackManager 等模块对应，避免孤立罗列概念。

2.1 引擎语言选择

2.1.1 H5 游戏引擎选型

浏览器 2D 游戏通常二选一：Phaser、PixiJS 这类渲染库，或 Cocos、Laya 这类带 IDE 的引擎。库轻，但场景、预制体、UI 和打包往往要自己拼；引擎把帧循环和输入包好了，更适合课设周期内做出能点的 Demo[2]。

本项目选用 LayaAir 3.x，理由如下：（1）课程与实验室已有 Laya 工程经验，环境搭建成本低；（2）3.x

对 TypeScript 支持与本项目语言一致；（3）2D 预制体、Area2D 与 UI2（GBox、GButton 等）可满足元菜单、跑图与战斗 HUD；（4）内置预览与发布至 bin/ 便于性能测试与答辩演示。Gregory 指出商业引擎将渲染、资源、场景更新与脚本生命周期统一管理；LayaAir 在 H5 场景下承担类似运行时平台的角色。

相对 Unity 、Unreal 等原生引擎，H5 分发成本更低，但受浏览器单线程与 GC约束更明显。Ullmann 等对引擎子系统耦合的研究表明，无论引擎种类，划分“引擎层—游戏逻辑层”都有利于维护[7]。因此 Laya 节点仅作表现层，核心玩法放在自研轻量 ECS 中更新，而非全部写入 Laya.Script 回调。

表2-1 主流 H5/跨平台方案与本项目选型对比

方案	优势	劣势	本项目
Phaser/PixiJS	轻量、社区活跃	工作流需自建	未采用
Cocos Creator	生态成熟、工具链完整	与现有课设栈不一致	未采用
LayaAir 3.x	IDE 一体、TS 原生、2D/UI2 完备	文档与社区小于头部引擎	采用

2.1.2 TypeScript与工程常量管理

脚本语言选用 TypeScript，在 JavaScript 上提供静态类型与模块化，实体、组件、系统增多时接口更稳定。Briand 等关于面向对象设计可维护性的实验表明，清晰模块边界与一致命名有助于降低理解成本[4]。可调战斗常量、UI 阈值、路径正则等静态配置集中在 src/defines/，经 index.ts 统一导出，避免业务文件顶部散落魔法数；策划数值外置 JSON 配表（见第 2.2 节），区分“工程常量”与“策划参数”。

2.1.3 主循环、预制体与引擎使用约束

主循环由 Laya.timer.frameLoop 驱动：每帧先 EcsWorld.update(deltaTime)，再由引擎渲染并分发 UI 事件，符合 Gregory 所述逻辑 tick 与绘制分离的常见做法[7,19],19]。入口 Main.ts 挂在 Area2D 子节点而非 3D 根节点，避免纯 2D Demo 进入 3D 渲染管线。

使用 Laya API 须统一 Laya. 前缀（如 Laya.Sprite、Laya.Handler.create）。角色与子弹经 Laya.Prefab.instantiate 生成。可加载资源（材质、贴图、预制体）须放在 assets/，不可放在 src/，否则预览路径解析失败。2D 位移须用 transform.position = new Laya.Vector3() 或 translate，不可对 position getter 返回的副本调用 setValue，否则逻辑坐标已变而画面不动——该问题在相机跟随与怪物追逐调试中多次出现，本文记录以供规避。上述主循环调用关系如图2-1所示。

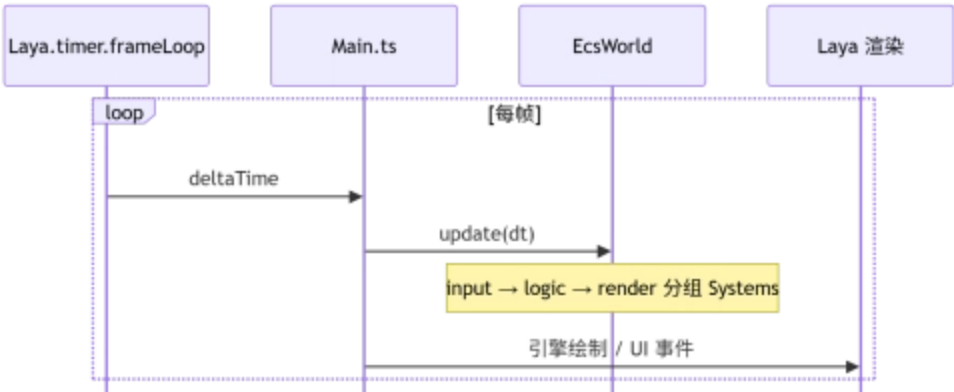




图2-1 主循环流程图

2.2 项目玩法架构选择

2.2.1 Survivor-like品类与核心循环

一局里玩家主要在躲、自动打、捡经验、升技能，怪会越刷越密。射击技巧让位给走位和构筑[7]。程序上要扛住三件事：移动时碰撞和结算不能飘；几分钟内多次升级，奖励得立刻写进属性或栏位；调数值尽量改表，别动战斗内核。

SimpleEcsDemo 默认创建玩家并由 PlayerAutoCastSystem 自动施法，ControlSystem处理走位，以降低操作门槛、突出构筑。经验由 ExperienceSystem 处理；击杀奖励与等级加成通过 defines 中 MONSTER_EXP_REWARD_BASE、MONSTER_EXP_REWARD_LEVEL_BONUS等常量调节，属于可配置节奏而非硬编码。

2.2.2 轻量实体组件系统（ECS）

实体组件系统（ECS）将对象拆为实体标识（Entity）、纯数据组件（Component）与无状态系统（System）。新增怪物行为时往往只需增 System 或扩展组件字段，无需维护大继承树；按类型批量处理更方便[7]。

Unity DOTS 采用 ECS + Job System + Burst，面向数万实体并行[7]。毕业设计周期与团队规模有限，完整DOTS 等价物学习与 Laya 节点绑定成本过高。本项目采用教学向轻量 ECS：EntityId 为数值；ComponentStore 按组件类型用 Map 存储；EcsWorld.update 单线程顺序调用已注册系统，按 input/logic/physics/render 分组排序。第 1 期目标是结构清晰、支撑数百同屏实体，不追求极致数据导向布局。

表2-2 Unity DOTS 与项目轻量 ECS 对比

维度	Unity DOTS	本项目 ECS
并行模型	Job System 多线程	单线程；追逐等片段可选Web Worker
内存布局	Archetype / Chunk Entities Graphics	按组件类型的 Map
与引擎绑定		ViewComponent 绑定Laya 节点
适用规模	数万实体级	数百实体级
迭代与调试成本	高	中低

玩法层组件含 PlayerTag/MonsterTag、Position/Velocity、Attribute（含可溯源 modifier）、Skill、SkillLoadoutState 等；系统含 MovementSystem、AttributeSystem、SkillSystem、BulletSystem、MonsterChaseSystem 等。FilterRegistry 提供 Players、Monsters 等命名筛选，避免各 System 重复写交集查询。怪物与玩家共享位移、属性、技能等组件，仅标签不同，体现组合优于继承。实体、组件与 Laya 表现节点的绑定关系如图2-2所示。





图2-2 ECS绑定关系图

2.2.3 数据驱动与JSON配表

Roguelite 与动作游戏普遍数据驱动：冷却、伤害、子弹参数、效果链外置为表，运行时加载[7]。常见做法包括：（1）运行时加载 JSON 并反射填充数据类；（2）构建期生成 TypeScript 常量；（3）注册表声明表结构、运行时按路径加载。本项目用第（3）种：

`tables.registry.json` 声明表名与文件，`ConfigBootstrap.ensureGameConfigLoaded()` 启动时加载，`initData` 将行挂到 `Data.Character`、`Data.Skill` 等命名空间。

一条技能对应一张 `Skill` 表和多行 `SkillEffect`。每行声明子弹、穿透/分裂/连锁修饰或直伤；`EffectExecutor` 从上到下执行，修饰行必须写在 `bullet` 前面。伤害写成“atk*1.5+10”这类字符串，由 `FormulaParser` 在白名单里算，禁用 `eval`。`Targeting` 负责自动索敌和简单指向。

JSON 方便版本管理和 diff，但错配往往集成时才暴露。启动用 `isGameConfigReady` 拦一道，公式和修饰器另有 `*.verify.ts` 脚本。

2.2.4 Roguelite跑图与关卡结构

元游戏采用 Slay the Spire 式 DAG 跑图：每局三个 Act，每 Act 为多层有向无环图，节点含休息、普通战斗、Boss、宝藏等[7]。地图生成一般含分层、节点抽样、连边约束与可达性校验；`RunMapGenerator` 实现固定行数网格、末行 Boss、中间随机分叉，`validateActReachBoss` 保证 Boss 可达。失败时回退保底图，避免进度卡死。

`SeededRng` 使相同种子可复现地图，便于答辩录像与回归。`MetaFlowController` 在用户确认节点后触发 `onEnterCombat`，将节点载荷（如 Boss 关更长生存时间）传入战斗场景，元游戏与局内逻辑松耦合。

2.2.5 战斗性能与密集实体仿真

Survivor-like 后期同屏怪物与弹幕增多，H5 热点包括：每帧大量碰撞、预制体反复实例化触发 GC、主线程追逐与分离计算。常见对策有对象池、纹理合批、逻辑暂停、纯数值计算迁至 Web Worker[15]。

`BulletPool`、`MonsterPool` 按预制体路径分桶复用节点；`BulletHitTest` 用距离平方减少开方；`GameSession.paused` 在技能面板打开时让战斗 `System` 早退。实体数达阈值时，`CombatDataPrepareSystem` 将坐标等快照经 `CombatDataBridge` 提交 Worker，在共享的 `combatWorkerLogic.ts` 中计算追逐、分离与摆动，主线程再写回速度；子弹碰撞依赖 Laya 节点与阵营判定，仍留主线程。Cantrell 等在 Unity 做物理人群疏散时同样强调重计算与交互渲染分离[12]；Helbing 等关于恐慌人群的研究提示密集场景须关注分离力与稳定性，与 `MonsterChaseSystem` 中分离项设计一致。

上述选型在可扩展与可测之间折中：ECS 与配表支撑内容迭代，池化与 Worker支撑同屏规模，跑图 DAG 支撑元游戏节奏。

2.3 UI架构选择

2.3.1 MVC分层与全屏界面栈

游戏 UI 常见模式有 MVC 、MVP 、MVVM 等[8]。Heijstek 等比较架构描述的图文形式，指出结构化图示有助于理解系统边界；本项目采用 MVC 变体：

Model：hp、装配状态等来自 ECS，跑图另有 RunMapPanelModel、RunMapState；
View：预制体和 UI2 节点只管摆位和显示；
Controller：UiControllerBase 子类管生命周期和路由，不碰战斗场景里的实体节点。
开始、选关、跑图都是全屏页。若多个面板一起 visible，容易挡操作、事件乱窜。UIStackManager 用栈管路由：push 压入并显示，pop 回到上一层，同一时刻只有一个全屏页拿焦点。

2.3.2 战斗内UI：HUD、暂停与技能装配

局内 UI 含主 HUD（血条、经验）、升级奖励面板、死亡重启面板与 Tab 技能装配面板。
SkillSelectPanelController 监听 Tab：打开时 GameSession.paused为真，战斗 System 在 update 入口早退，怪物与子弹逻辑冻结，UI 仍接收指针事件——即输入层与战斗逻辑层分离，避免面板打开仍被攻击。

技能装配用长按拖拽：SkillDragService 按下超过阈值后创建顶层代理图标，规避 GBox 裁剪导致的命中错误；SkillSlotHitTest 统一坐标系判断落点属于装备栏或效果槽。关闭面板时，SkillLoadoutSyncSystem 将 SkillLoadoutState 原子写回Skill 与 PlayerAutoCastSystem，避免半状态仍施放旧技能。图标由配表 iconPath 与 SkillIconBinder 加载，资源目录约定文件名与 id 对应。

跑图界面 RunMapPanelView 按 DAG 状态渲染节点与连线，MapNodeIconUtil、LineLayoutUtil 负责布局；玩家只能选与当前节点相邻且已解锁的边，规则由模型层校验，不在 View 硬编码。

2.3.3 UI架构选型的综合评价

UI 架构选型归纳见表 2-3。界面不持有玩法真相，玩法状态以 ECS 与配表为准，UI 经 Controller 与 SyncSystem 读写；全屏流程用栈管理，局内装配用暂停位切断战斗 tick。该划分降低循环依赖，也使第 7 章功能测试（Tab 暂停、装配同步、跑图可达）有明确验收口径。

2.4 本章小结

本章从引擎语言、玩法架构、UI 架构三方面说明 LayaAir + TypeScript、轻量ECS + 配表 + Roguelite 跑图、MVC + UI 栈的技术路线及取舍，将在第 4 章落实为 ECS/MVC/Worker 设计，在第 5-7 章经实现与测试验证[17]。

表2-3 UI 架构选型要点汇总

问题	选型	理由
全屏导航	UIStackManager 路由栈	避免多面板同显
局内构筑	Tab + 拖拽 + Sync- System	暂停战斗、原子同步
数据绑定	预制体 View + Con- troller	便于美术替换与动画

第三章 系统需求分析

本章在第 2 章技术选型基础上，给出 Survivor And Fight 的可行性论证、功能与非功能需求及验收口径，为第 4 章总体设计与第 7 章测试提供依据。需求描述以可验证为原则：每条 Must 级需求均可在代码或测试用例中找到对应实现或断言。

3.1 可行性分析

3.1.1 技术可行性

代码侧已有 LayaAir 3.x + TypeScript 工程：轻量 ECS、配表战斗、元菜单、DAG跑图、技能装配 UI，以及对象池和 Worker。引擎带 2D 预制体和 frameLoop；答辩用Chromium 支持 WebGL2 与 Worker。文献里 SyDRA 讨论子系统边界[6]；Cantrell等说明在商业引擎上叠领域逻辑可行[12]。

已具备：ConfigBootstrap.ensureGameConfigLoaded 用 fetch 和 Laya.loader双路读表；isGameConfigReady 挡住未就绪就进战斗；ECS、公式、修饰器有 verify.ts。风险：千怪帧率、Worker 与主线程是否算同一套、UI2 拖拽命中——五波实验和 T- F-05/06 已部分覆盖。

结论：课设周期内能交原型和论文，路线可行。

3.1.2 经济与社会可行性

作为教学原型，主要成本为开发与测试人力，无服务器与支付。游戏为奇幻战斗题材，须控制暴力表现并提示合理游戏时长；跑图休息节点提供节奏缓冲。不采集用户敏感信息，符合单机演示定位。

3.2 用户角色与运行环境

3.2.1 用户角色

系统仅设玩家单一角色：浏览主菜单、选关或跑图、局内移动与构筑、死亡后重启。无管理员后台；未来若扩展存档，仍保持“玩家”为唯一角色。

表3-1 运行环境建议配置

类别	要求
处理器	四核 2.0 GHz 及以上
内存	8 GB 及以上
显卡	支持 WebGL2
浏览器	Chrome / Edge 最新稳定版（答辩演示以 IDE 内置 Chromium 为准）
开发环境	LayaAir IDE 3.x 、Node.js（工具脚本）、TypeScript

3.2.2 运行环境

3.3 功能性需求

表3-2汇总主要功能模块；详细条目见第 3.4 节。

表3-2 主要功能性需求一览

模块	需求描述
ECS 核心玩法实体	创建/销毁实体；按类型存取组件；分组更新
技能战斗	玩家/怪物标签；移动、视图同步；hp/maxHp 与 modifier
怪物系统	多 effect 技能；自动施法；子弹表驱动伤害与穿透/分裂/连锁
怪物系统	波次刷怪；追逐与分离；接触伤害；离屏回收
进度系统	经验、升级、随机奖励
元游戏跑图	开始/选关；onEnterCombat；生存计时胜利
技能装配 UI	三幕 DAG；节点类型；种子可复现；Boss 可达
配置	Tab 暂停；三装备栏；长按拖拽装配
	JSON 注册加载；缺失时 DEFAULT_* 与去重日志

3.3.1 用例：局内战斗

参与者：玩家。前置条件：isGameConfigReady()==true，SimpleEcsDemo.init()完成。主成功场景：（1）输入移动；（2）PlayerAutoCastSystem 自动施法；（3）子弹碰撞并应用穿透/分裂/连锁；（4）击杀获经验并升级；（5）Tab 打开技能面板， GameSession.paused=true，装配后恢复。扩展：hp≤0 时显示重启面板。对应第 5.5 节场景一、二。

3.3.2 用例：元游戏进入战斗

用户在跑图或选关界面确认节点，MetaFlowController 触发 onEnterCombat(payload)； Main 显示战斗容器并 demo.init()，CombatSurvivalTimer 启动；Boss 节点载荷可携带更长胜利时间。

3.4 功能需求详细说明

3.4.1 ECS核心

须提供实体创建/销毁 API，销毁时移除全部组件；同类型组件不可重复挂载；系统按 input/logic/render 分组与优先级确定性调度；支持按组件类型遍历及命名筛选器查询[7]。

3.4.2 战斗与技能

玩家须具备 hp/maxHp 与可溯源 modifier。技能由配表定义冷却与 effect 列表；至少三个装备栏独立冷却。效果类型覆盖 bullet、modifier_*、direct_damage；子弹带阵营，仅对敌对阵营生效；连锁半径有界，避免无界遍历[9]。

3.4.3 怪物、元游戏与UI

怪物在玩家周围环带生成并保持最小间距；须实现追逐、接触伤害、离屏回收；击杀给予经验（含等级加成）。跑图每局三幕 DAG，仅沿已解锁邻接边前进； validateActReachBoss 保证可达。全屏界面经 UIStackManager；Tab 暂停战斗；拖拽区分技能/效果槽；血条与升级面板实时反馈[17]。

3.5 非功能性需求

性能： 目标稳定 60 FPS；同屏规模由 MONSTER_COUNT 与波次控制；对象池与Worker 按第 4.2.3 节启用。可维护性：模块边界与第 4 章一致；静态常量集中于 defines。

可靠性: Worker/配表/跑图失败时的回退路径见第 4.4 节。
兼容性: 优先 Chromium; Worker 不可用时 computeSync 主线程回退[5]。

表3-3 非功能需求量化指标（验收参考）

指标	目标值	测量方法
战斗帧率（常规波次）	均值 $\geq$ 55 FPS	TestLevelFpsTracker
千怪消融 P95	池化后显著低于无池	表 7-6
配表就绪	Data.Skill 非空	isGameConfigReady
Worker 行为	与主线程无逻辑漂移	轨迹对比（见第 7 章）

3.6 数据、接口与约束

逻辑数据实体: 运行时 ECS 组件; 配表行 (Character、Skill、SkillEffect、Bullet); RunMapState 图结构; GameSession、SkillLoadoutState。

关键接口: (1) ensureGameConfigLoaded(): Promise<boolean>; (2) onEnterCombat(payload ControlInputSource; (4) UIStackManager.push/pop; (5) CombatWorkerComputeRequest/Response 字段版本兼容。

约束与假设: 浏览器启用 JavaScript 与 WebGL; 开发阶段已执行 sync-config-to-bin.ps1; 单机本地运行; 2D 俯视角; 同屏数百实体级, 非 MMO 规模。

3.7 需求优先级

表3-4 需求优先级 (MoSCoW)

级别	需求
Must	ECS 战斗、子弹碰撞、移动、刷怪、配表、主菜单进战斗、生存胜利
Should	跑图 DAG、技能装配、升级奖励、对象池与 Worker
Could	法杖三槽完整 UI、全量 Effect 独立特效
Won't	联机、账号、支付 (本期)

3.8 需求与测试映射

表3-5列出需求与验证手段的对应关系, 供第 7 章追溯[18]。

表3-5 需求与测试映射 (节选)

需求	验证方式	编号/脚本
ECS 实体销毁清理组件	单元验证	ecsCorePhase1.verify.ts
Tab 暂停与装配同步	手工用例	T-F-05、T-F-06
Boss 可达	生成断言	validateActReachBoss
公式安全解析	单元验证	formulaParser.verify.ts

3.9 本章小结

本章写完可行性和需求清单，并挂上 MoSCoW 与测试映射。下一章展开分层设计和 Worker、可靠性细节。

第四章 系统总体设计

4.1 设计原则

系统总体设计遵循以下原则，并在后续模块实现中反复体现：

- (1) 数据与逻辑分离：组件仅存数据，系统执逻辑；可调参数尽量外置 JSON 配表，工程常量集中于 src /defines/[7]。
- (2) 单一职责：BulletSystem 负责子弹运动与碰撞；EffectExecutor 负责配表效果解释；MetaFlowController 仅协调元游戏流程，不直接改写 ECS 组件[4]。
- (3) 可验证：核心模块提供公式解析、ECS 核心、属性修饰器等单元验证脚本，变更后可快速回归[18]。
- (4) 渐进复杂度：先做轻量 ECS 和单线程 tick；池化和 Worker 只在怪够多时再开。
- (5) 低耦合：UI 经 Controller 、SyncSystem 读写 ECS ；Worker 与主线程共用一份combatWorkerLogic.ts [17]。
- (6) 可降级：表没加载、Worker 起不来、地图生成失败时，都有回退（第 4.4 节、第 4.5 节）。

4.2 系统具体设计

逻辑上分五层：表现、UI 控制、游戏逻辑、领域服务、基础设施，见图4-1。Main.ts在 Laya.timer.frameLoop 里调 EcsWorld.update，随后引擎画画面、发 UI 事件。菜单、选关、跑图和战斗在同一工程里，靠容器显隐和 MetaFlowController 切换，不必维护两套入口。

4.2.1 基于ECS的Gameplay实现

局内玩法走 ECS 。EntityId 是数字；ComponentStore 用 Map 按类型存组件；EcsWorld 按 input/logic /render 分组调 System。Position、Velocity、Attribute、Skill 等只放数据；MovementSystem、BulletSystem 等遍历实体改字段，本身不记状态。



图4-1

标签与筛选：玩家与怪物共享位移、属性、技能等组件，仅通过 PlayerTag/MonsterTag区分。FilterRegistry 提供 Players、Monsters 等命名筛选器，避免各 System 重复编写交集查询。该设计体现组合优于继承：新增怪物行为时扩展组件字段或新增System，而非维护 MonsterBase 类树。

战斗数据流：每帧 CombatDataPrepareSystem 采集坐标等快照 → 可选 CombatDataBridge提交 Worker 执行 processCombatFrame → 主线程 MonsterChaseSystem 应用追

逐/分离结果 → BulletSystem 在主线程完成碰撞（依赖 Laya 节点与阵营判定）。

GameSession.paused 在死亡、技能 Tab 或胜利时冻结逻辑 System 的 update 入口。

配表驱动：Skill 与多行 SkillEffect 描述冷却与效果链；EffectExecutor 按序解释 bullet、modifier_*、direct_damage。修饰器须位于后续 bullet 之前，形成链式组合语义。伤害字符串由 FormulaParser 在白名单运算下解析，禁止 eval。

4.2.2 基于MVC的UI结构实现

元游戏与局内 UI 采用 MVC 变体[17]：

Model：hp、装配来自 ECS；跑图用 RunMapPanelModel、RunMapState；

View：预制体、UI2 节点负责布局；

Controller：UiControllerBase 子类管生命周期和路由，不得直接改战斗实体。

全屏页走 UIStackManager：push 显示、pop 返回，始终只有一个全屏页在前台。MetaMenuBootstrap 注册开始、选关、跑图路由。

局内 Tab 由 SkillSelectPanelController 监听：打开时 GameSession.paused=true，战斗 System 不 tick，UI 仍能吃点击。拖拽走 SkillDragService，落点用 SkillSlotHitTest；关面板时 SkillLoadoutSyncSystem 一次性写回 Skill 和 PlayerAutoCastSystem。

跑图连线由 RunMapPanelView 画，能不能走由 Model 判，View 不写规则。

核心约束：界面不拥有玩法真相——玩法状态以 ECS 与配表为准，UI 仅经 Controller/SyncSystem 读写。

4.2.3 基于Web Worker和对象池的优化实现

对象池：BulletPool、MonsterPool 按预制体路径分桶复用 Laya 节点，抑制instantiate/destroy 触发的 GC 尖峰。MonsterRecycleSystem 在怪物远离玩家超过阈值后回收实体，维持活跃集合规模。

Web Worker：当场上实体数 $\geq$ COMBAT_WORKER_MIN_ENTITY_COUNT 时，CombatDataPrepareSy将快照经 CombatDataBridge 发送至 Worker；combatWorkerLogic.ts 在主线程与Worker 间共享，计算追逐、分离与摆动。CombatDataBridge.prepareFrame 优先消费异步结果，失败或未就绪时调用 computeSync 回退。子弹碰撞、连锁与分裂仍保留主线程，因依赖引擎节点与即时命中判定。

逻辑暂停：GameSession.paused 在技能面板、死亡、胜利时让战斗 System 直接 return，不额外分配资源，思路与 Cantrell 等把重算和画面拆开一致[5]。

4.3 系统交互与时序

图4-2描述从启动、进战斗、切元游戏到重启的调用顺序，由 figures/system-sequence.mmd经 build.ps1 导出为 PNG。

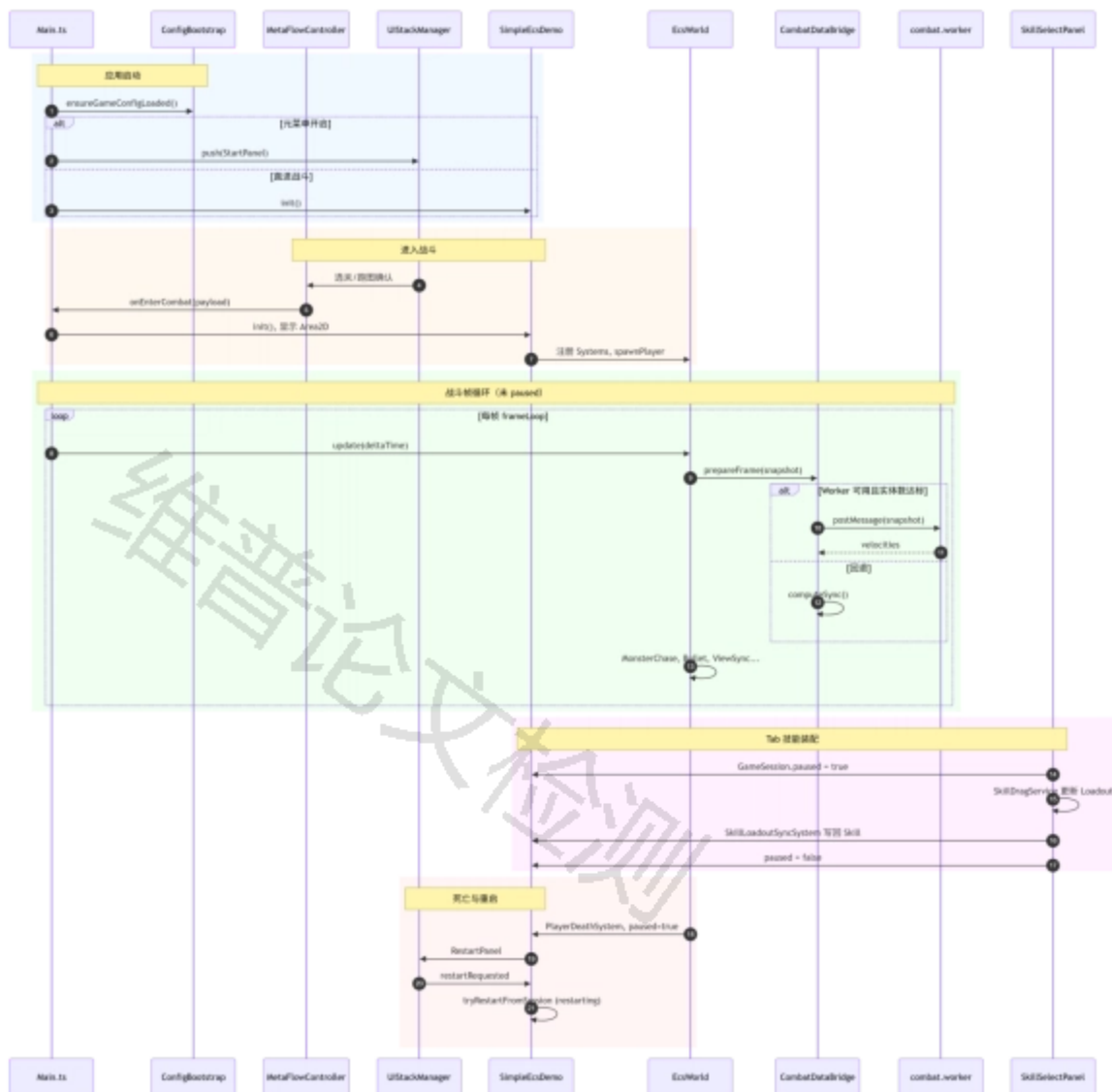


图4-2 系统主要交互时序

4.4 可靠性设计

配表坏了、Worker 起不来、地图生成失败时，系统应尽量还能玩或至少能调试，而不是白屏退出。做法如下。

(1) 配表加载与就绪判据。ConfigBootstrap.ensureGameConfigLoaded 遍历configBases(), 依次尝试 fetch 与 Laya.loader 双通道加载 tables.registry.json及各表; isGameConfigReady() 要求 configLoaded 且 Data.Skill 非空, 作为“可否进入战斗”的全局闸门。Main.onStart 与 SimpleEcsDemo.spawnPlayer 均在生成实体前等待或复查该状态。

(2) 运行时缺表降级。SimpleEcsDemo 维护missingConfigLogged:Set<string>, 通过 logMissingConfigOnce 对每个缺失键仅告警一次, 避免刷屏。defines 中 DEFAULT_PLAYER_HP DEFAULT_PLAYER_PREFAB、DEFAULT_PLAYER_AUTO_SKILL_ID 等在查表失败时回退; 升级奖池表为空时返回内联默认条目, 保证演示不中断。

(3) Worker 双路径。CombatDataBridge.initWorker 失败、onerror 或 postMessage异常时将 workerUsable=false; prepareFrame 无可用异步结果时调用 computeSync,

与 Worker 共用 processCombatFrame。MonsterChaseSystem 无 Worker 结果时走本地追逐 + 分离；BulletSystem 在存在连锁弹等 Worker 未覆盖场景时回退完整主线程逻辑。实体数低于 COMBAT_WORKER_MIN_ENTITY_COUNT 时不提交 Worker，减少通信开销。

(4) 跑图生成保底。RunMapGenerator.generate 在最多 RUN_MAP_GENERATION_MAX_RETRIES次随机尝试后，若 validateActReachBoss 仍失败，则调用 generateFallback生成三幕各「休息→ 战斗→ Boss」线性图，保证 Boss 可达、进度不卡死。

(5) 重启防重入。RestartPanelSystem 将 session.restartRequested 置真；SimpleEcsDemo.tryRestartFromSession 在 restarting=true 期间异步清场与重生，update 中避免重复触发，finally 复位标志，防止监听器与实体重复注册。

(6) 预制体加载失败。实例化失败时 createPlaceholderNode() 生成占位碰撞体，便于在无美术资源环境下继续调试逻辑。

4.5 容错与降级策略

表 4-1 汇总主要故障模式与系统响应，与第 4.4 节实现一一对应。

死亡和 Tab 装配共用 GameSession.paused 冻战斗逻辑。RestartPanelSystem用 isPlayerDead() 决定弹不弹重启，免得装配面板误进重启。

表4-1 容错与降级策略

故障模式	检测方式	降级行为
配表 JSON 404或空表	isGameConfigReady()==false	控制台报错提示；玩家/技能使用 DEFAULT_*；自动施法可能退化
Worker 初始化/通信失败	workerUsable=false	computeSync 主线程重算追逐
实体数低于Worker 阈值	快照计数	不投递 Worker，全主线程
跑图随机生成不可达	validateActReachBoss	generateFallback 线性保底图
预制体路径无效	instantiate 异常	占位节点 + 日志
UI 路由加载失败	Controller 构造异常	跳过该路由并日志，不阻断战斗容器

4.6 可扩展性设计

本系统的可扩展性建立在 ECS 的组合优于继承原则之上，而非为每种怪物或技能派生子类。

实体扩展：新玩法维度优先新增组件类型（如未来的 BuffComponent、ShieldComponent），再在独立 System 中读写；已有 MonsterTag 实体自动参与FilterRegistry.Monsters，无需修改 EcsWorld 内核。

行为扩展：新行为对应新 System 或扩展现有 System 的分支；例如新增“光环”只需注册 AuraSystem 并在 SimpleEcsDemo 构造函数中 addSystem，不必改动MovementSystem。

技能扩展：新 effect 类型在 EffectExecutor 增加分支；修饰器仍通过“写上下文 → 后续 bullet 消费”扩展，策划在 SkillEffect 表调序即可组合。新子弹形态仅需配表 Bullet 行与预制体路径。

内容扩展：新跑图节点类型扩展 RunNodeType 与 RunMapGenerator 权重表；新全屏界面在 UIStackManager 注册路由与 Controller，不修改战斗 ECS。

与继承方案对比：若采用 Player extends Character extends Actor，每增加一种“会分裂的精英怪”都需新建子类并重写 update，导致 Laya 节点与逻辑深度耦合。当前方案下，精英怪仅是 MonsterTag + 额外组件（如更高 Attribute 或专用 MonsterDef.waveWeight），行为差异由 MonsterAutoCastSystem 等组合完成，符合 Gregory 与 ECS 文献对“数据导向、增量扩展”的论述[7]。

4.7 本章小结

本章给出设计原则、分层架构，并从 ECS Gameplay、MVC UI、Worker/对象池三方面说明系统具体设计；进一步给出端到端时序、可靠性机制、容错表与基于组合的可扩展性策略，为第 5 章实现细节提供结构框架。

第五章 核心模块详细设计与实现

本章按第 4 章总体设计，结合 src/ 源码说明核心模块实现。SimpleEcsDemo 为组合根（Composition Root），集中注册系统、池与 UI 控制器，是阅读仓库的首选入口。

5.1 ECS架构GamePlay实现

5.1.1 世界、组件与筛选器

EntityManager 使用自增 ID 与空闲列表复用实体。ComponentStore 按类型维护 Map<EntityId, Component>。EcsWorld.update 按 input/logic/render 分组顺序调用系统[7]。FilterRegistry 在构造时 registerNamedFilters，提供 Players、Monsters 等命名筛选。

5.1.2 属性、移动与自动施法

AttributeSystem 支持按来源增删 modifier，供 Buff 与升级奖励使用。MovementSystem、ControlSystem 处理位移与输入。PlayerAutoCastSystem 对三装备栏维护独立冷却与 pendingCasts；MonsterAutoCastSystem 驱动怪物施法。

5.1.3 技能与效果执行链

PlayerAutoCastSystem 决定施放后，流程为：（1）从 SkillLoadoutState 读取技能 id 并检查 Skill.cooldownRemain；（2）CastPlan 按配表顺序解析 effect 行；（3）EffectExecutor 分发：modifier_* 写入分裂/连锁/穿透上下文，供下一条 bullet 消费；（4）bullet 调用 BulletSystem.spawnBulletWithOptions；（5）direct_damage 经 FormulaParser 扣血。具体执行过程如图5-1。

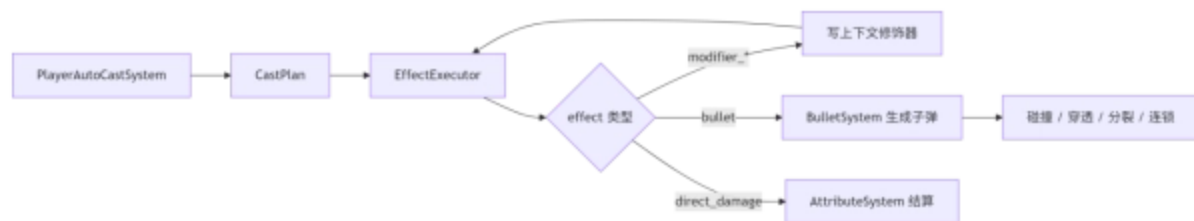


图5-1 技能效果执行链

表5-1 效果类型与执行器映射

effect 字段	行为
bullet	BulletSystem.spawnBulletWithOptions
modifier_split	设置分裂数量
modifier_chain	设置连锁与半径
modifier_pierce	增加穿透
direct_damage	公式直伤

5.1.4 子弹碰撞与怪物AI

BulletSystem 从 BulletPool 取节点，用 hitRadiusSq 做圆碰撞；命中后扣血、递减穿透，并按配置处理连锁与分裂[9]。MonsterChaseSystem 计算追逐方向，叠加分离力与随机摆动；有 Worker 结果时 applyWorkerVelocities，否则本地计算。 MonsterWaveSpawnSystem 在环带刷怪； MonsterRecycleSystem 与 MonsterPool 回收离屏单位。 MonsterContactDamageSystem 处理接触伤害。

5.1.5 经验、升级与元游戏数据

ExperienceSystem 在击杀时增加经验并判断升级。 UpgradeRewardSystem 打开奖励面板； RewardPoolService 按权重抽取条目， RewardApplyService 写回属性或技能。 RunMapGenerator 生成三幕 DAG，失败时 generateFallback。 MetaFlowController协调 onEnterCombat 并将节点载荷传入战斗场景。

5.2 MVC架构UI实现

5.2.1 UI栈与元游戏界面

MetaMenuBootstrap 向 UIStackManager 注册 StartPanel、 SelectLevelPanel、 RunMapPanel 等路由。用户确认节点后触发 onEnterCombat， Main 隐藏菜单容器、显示战斗 Area2D 并 demo.init()。 RunMapPanelView 使用 MapNodeIconUtil、 LineLayoutUtil渲染 DAG；仅允许选择已解锁邻接边。

5.2.2 战斗HUD与技能装配

MainHudSystem、 BloodBarSyncSystem 将 ECS 中 hp 映射至 ProgressBar。 SkillSelectPanelCo绑定 MainUIPanel.lh， Tab 切换时设置 GameSession.paused。 SkillDragService 在超过 LONG_PRESS_MS 后使用顶层代理图标拖拽； SkillSlotHitTest 判断装备栏/效果槽落点。 SkillLoadoutSyncSystem 关闭面板时原子写回 Skill 与 PlayerAutoCastSystem，并调用 getCombatEffectIds 刷新可战斗 effect 集合。

5.2.3 输入与重启面板

InputService 封装键盘/指针； ControlInputAdapter 转为 ControlSystem 可读方向。 Tab 打开面板时战斗 Control 不更新， UI 仍可接收事件[17]。 PlayerDeathSystem在 $hp \leq 0$ 时置 paused； RestartPanelSystem 经 UI 栈显示重启面板； tryRestartFromSession在 restarting 保护下清场重生。

依赖关系保持单向： UI → ECS； Systems → 配表；禁止 System 直接操作 UI 节点。



图5-1 Tab 技能装配交互时序 (Mermaid)

5.3 Web Worker和对象池的优化实现

BulletPool、MonsterPool 按预制体路径分桶，acquire 优先复用空闲节点，不足时再 instantiate。BulletHitTest 使用距离平方减少开方。

CombatDataPrepareSystem 在 isPaused() 为假时打包怪物坐标等快照，调用CombatDataBridge.prepareFrame。桥接层在 Worker 就绪且实体数达标时postMessage，否则 computeSync；MonsterChaseSystem 与 BulletSystem 按是否有 Worker 结果及是否存在连锁弹，选择应用速度或走完整主线程碰撞。性能消融可通过 MetaRunSession.testUseCo与 testDefine 预设控制（见第 6 章）。

5.4 游戏主循环设计

5.4.1 入口与配置时序

Main.onStart 创建 InputService、ControlInputAdapter、SimpleEcsDemo，注册 frameLoop，并 await ensureGameConfigLoaded()。配表优先于实体生成，避免Data.Character.GetByID 为空。若 META_MENU_ENABLED 为真则 startMetaMenu，否则直接 demo.init()。

5.4.2 SimpleEcsDemo构造与init

构造函数内：创建 FilterRegistry；注册 AttributeSystem、ExperienceSystem、BulletSystem（注入池与桥接）、CombatDataPrepareSystem、SkillSystem、MonsterWaveSpawnSyst UpgradeRewardSystem、MainHudSystem、SkillSelectPanelController 等。init()在配表就绪后按 Character 行加载玩家预制体，挂载 PlayerTag、Position、Attribute、Skill、SkillLoadoutState、ViewComponent 等；怪物由波次系统动态生成。

5.4.3 帧循环与暂停

每帧：若 restarting 则跳过重启检测；否则 tryRestartFromSession；若 isPaused()则战斗 System 早退；否则 world.update(deltaTime)。生存计时与 Tab 暂停在 Main层联动，ECS 层以 GameSession.paused 为唯一逻辑冻结开关。

5.5 典型场景实现剖析

5.5.1 场景一：玩家首次进入战斗

步骤 1: Main.onStart → ensureGameConfigLoaded。步骤 2: 元菜单 UIStackManager.push(St战斗容器暂隐

藏。步骤 3：选关/跑图 → onEnterCombat → demo.init(), CombatSurvivalTimer启动。步骤 4：首帧 ControlSystem、PlayerAutoCastSystem、MonsterWaveSpawnSystem协同运行。对应测试 T-F-01 、T-F-09。

5.5.2 场景二：Tab打开技能面板并更换装备

Tab → paused=true → 长按拖拽 → SkillLoadoutSyncSystem 写回 → paused=false，下一冷却周期生效新技能。对应 T-F-05 、T-F-06。

5.5.3 场景三：怪物密集时的Worker卸载

实体数 ≥ 阈值时 CombatDataPrepareSystem 提交 Worker；主线程仍处理子弹碰撞。关闭 Worker 或降低上限后 P95 帧时间上升（见第 7 章），说明优化有效且边界清晰。

5.6 关键代码文件索引

表5-2 核心源码文件索引

模块	路径
入口	src/Main.ts
ECS 世界	src/ecs/core/World.ts
组件存储	src/ecs/core/ComponentStore.ts
战斗 Demo	src/game/demo/SimpleEcsDemo.ts
子弹	src/game/bullet/BulletSystem.ts
子弹池	src/game/bullet/BulletPool.ts
效果执行	src/game/skill/EffectExecutor.ts
Worker 桥接	src/game/combat/CombatDataBridge.ts
Worker 逻辑	src/game/combat/combatWorkerLogic.ts
跑图生成	src/game/run/RunMapGenerator.ts
元流程	src/game/meta/MetaFlowController.ts
UI 栈	src/game/ui/mvc/UIStackManager.ts
技能面板	src/game/ui/skillselect/SkillSelectPanelController.ts
配表引导	src/config/ConfigBootstrap.ts
静态常量	src/defines/*.ts

5.7 功能模块与论文章节对照

表5-3 功能模块与论文章节对照

功能模块	论文章节
ECS Gameplay（实体、技能、子弹、怪物）	第 5 章第 5.1 节；第 4 章第 4.2.1 节
MVC UI（栈、跑图、技能装配）	第 5 章第 5.2 节；第 4 章第 4.2.2 节
Web Worker 与对象池	第 5 章第 5.3 节；第 6 章

主循环与入口	第 5 章第 5.4 节
可靠性、容错与可扩展性	第 4 章第 4.4–4.6 节
测试与性能实验	第 7 章

5.8 本章小结

本章按 ECS Gameplay 、MVC UI 、Worker/对象池、主循环四条主线说明了实现细节，并以三个典型场景串联配置—流程—战斗的贯通路径。实现中显式采用组合根（SimpleEcsDemo）、对象池、桥接（CombatDataBridge）、策略（Targeting）、状态（GameSession.paused）等模式，未引入过度抽象。

主要难点与对策包括：（1）配表未同步至 bin/config 导致 404——以 sync-config-to-bin.ps1与 isGameConfigReady 应对；（2）UI2 拖拽命中——顶层代理与 SkillSlotHitTest；（3）Effect 链顺序——文档与样例配表约束；（4）Worker 与主线程逻辑一致——共享combatWorkerLogic.ts 与 computeSync 回退。表 5-2、表 5-3 便于评阅对照源码。下一章专述性能优化与实测数据。

第六章 性能优化与工程实践

本章对照第 4.2.3 节设计，写清已落地的优化、实验步骤和工程习惯，并与第 7 章数据对齐。原则是先量后开：池和 Worker 都能关，方便对照和排错。

6.1 性能目标与瓶颈

目标：1920×1080、IDE 内置 Chromium 预览下，普通波次平均帧率贴近 60 FPS；千怪时 P95 帧时间能压下去。

瓶颈：追逐和子弹碰撞占 CPU；预制体反复创建触发 GC；主线程还要跑脚本和绘制[7, 5], 5]。人群仿真文献也主张把重算和画面分开[3, 5], 5]，本项目用池化、Worker 和 paused做同类处理。

6.2 对象池

BulletPool、MonsterPool 按 prefabPath 分桶。acquire 先取空闲节点并重置，不够再 instantiate；put 在子弹打完或 MonsterRecycleSystem 回收时调用。

池化把“每帧猛建猛删”变成“高峰过后反复用”，GC 尖刺减轻，P95 更好看。第 7 章第 4–5 波同窗对比：开池后 P95 从 76.50 ms 到 48.60 ms（约降 36.5%），平均FPS 几乎不变，说明卡的是尖峰不是均值。

6.3 Web Worker卸载

追逐、分离、摆动只有数字，没有节点。CombatDataPrepareSystem 在实体数 $\geq$ COMBAT_WORKER_MIN_ENTITY_COUNT 时打包快照，CombatDataBridge 丢给 Worker 跑processCombatFrame。主线程和 Worker 共用 combatWorkerLogic.ts；prepareFrame不行就 computeSync。

子弹碰撞、连锁、分裂离不开 Laya 节点，仍主线程算（见第5.1 节）。第 1–3 波在实体达标时启用 Worker 参与追逐计算；第 4–5 波用于对象池同窗对比（第 5 波关闭 Worker，以隔离池化对 P95 的影响，见第 7 章）。

6.4 渲染与逻辑侧其它优化

碰撞：BulletHitTest 用距离平方，少做开方。

暂停：GameSession.paused 时战斗 System 直接 return ，Tab、死亡、胜利都算。

图集：接上 AtlasConfig.atlascfg 和 TextureAtlasService 后，千怪千弹波 FPS 从 13.32 到 15.10（约 +13.4%），见表 7-4。

配置：策划表启动读一次；工程常量放 defines，热路径不碰磁盘。

6.5 帧预算与内存

60 FPS 时一帧大约 16.67 ms。粗分：输入和 UI 1-2 ms，ECS（含子弹）6-8 ms，Worker 通信 1 ms 以内，其余给渲染[7]。超了先开池或 Worker，再降同屏上限，别在业务里硬砍分辨率。

短生命周期对象多会诱发 GC。池化把分配摊平，Performance 面板里 Scripting/GC 段往往跟着好看些。观测靠 TestLevelFpsTracker；missingConfigLogged 对缺表键只打一次日志。

6.6 性能实验设计

6.6.1 实验目的与变量

看池、图集、Worker 对 FPS 和 P95 的影响[7, 10], 10]。

自变量：池开/关；Worker 开/关（怪够多时）；同屏怪数。

因变量：平均 FPS、P95（消融段）。

控制：同一浏览器、1920×1080、Level3 五波脚本、关后台大程序。

6.6.2 实验步骤

1. IDE 预览或发布，可选开 Performance；
2. Level3：5 波各 10 s，怪数 10/100/1000，第 4-5 波千怪消融；
3. TestLevelFpsTracker 记均值，第 4-5 波记 P95；
4. 对照表 7-5、表 7-6 并写几句解读。

6.6.3 优化自检清单

合代码前自查：热路径有没有狂 new；销毁前有没有 pool.put；碰撞是不是平方距离；paused 能不能挡住无关 System；Worker 载荷是不是纯数；配表是不是只加载一次。

6.7 绿色计算与兼容性

CPU 平均占用低一点，笔记本续航会好一些：池减分配，暂停减空转，MONSTER_COUNT 别无脑拉满。Worker 挂了走 computeSync；配表 fetch 和 Laya.loader 双路；没

Worker 的浏览器也能跑主流程。换浏览器或引擎版本后，要回归 Worker 脚本路径和 bin/config。

6.8 本章小结

本章写了池、Worker、碰撞/暂停/图集和帧预算，并给出实验步骤。数据上：池主要救 P95；图集抬千怪波平均 FPS；Worker 在第 1-3 波参与追逐重算。下一章列用例、环境和读表方法。

第七章 系统测试与验证

本章对 Survivor And Fight 开展单元验证、功能回归与性能实验，验证第 3 章需求与第 4 章设计是否落地。测试策略遵循 Petty 等对仿真与交互系统 V&V 的要求[16]：核心契约用脚本验证，玩法闭环用手工用例，性能用可复现关卡与日志统计。

7.1 测试策略

单元验证：ECS 核心、公式解析、属性修饰器——变更相关模块后须重跑对应*.verify.ts。集成/系统测试：跑图可达性、Tab 装配同步、Worker 轨迹一致性——手工或脚本断言。性能测试：Level3 五波脚本 + TestLevelFpsTracker，对比表 7-5 、7-6。回归清单：修改 FormulaParser、ComponentStore、AttributeSystem、配表、CombatDataBridge后，按附录需求一测试矩阵或表 3-5 回归主路径。

7.2 单元验证

7.2.1 ECS核心

ecsCorePhase1.verify.ts 覆盖：实体 ID 唯一；组件增删与 GetComponent；销毁后不可访问；多 System 按分组与 priority 顺序执行[18]。

7.2.2 公式解析器

formulaParser.verify.ts 覆盖：四则运算、括号、属性别名； 非法字符与边界公式（以实现为准的安全返回/抛错）。配表伤害依赖解析器，任何变更须 rerun。

7.2.3 属性修饰器

attributeModifier.verify.ts 覆盖：多 modifier 叠加；按 sourceId 批量移除； getModifierContributions 可追溯。升级奖励与 Buff 均依赖该机制[4]。

7.3 功能测试

表 7-1 列出主要手工用例，走通战斗、元游戏、跑图、装配和重启。

表7-1 功能测试用例摘要

编号	步骤	预期结果
T-F-01	启动并等待加载	isGameConfigReady，技能表非空
T-F-02	进入战斗移动	角色与相机正常
T-F-03	怪物接近	追逐、扣血、血条更新
T-F-04	观察自动攻击	子弹命中；穿透/分裂/连锁
T-F-05	Tab 打开技能面板	paused=true，可拖拽装配
T-F-06	关闭面板	战斗恢复，下一冷却周期新技能生效
T-F-07	升级	奖励面板与应用
T-F-08	玩家死亡	重启面板，restarting 无重入
T-F-09	元游戏进战斗	生存计时启动
T-F-10	Boss 节点	Boss 胜利时长参数生效

7.3.1 集成测试补充

跑图可达性：对 RunMapGenerator.generate 多次随机种子断言 validateActReachBoss；失败应走 generateFallback。

装配同步：互换装备栏技能后，下一冷却周期伤害/图标与装配一致，否则 SkillLoadoutSyncSystem存在缺陷。

Worker 一致性：Worker 开、关各跑一段，怪物轨迹可差一帧，但算法不能两套[10]。

7.4 性能测试

7.4.1 测试环境

表 7-2 为五波实验与功能抽测的统一环境（2026-05-18，单次运行）。

选关 Level3 共 5 波：第 1-3 波怪物 10/100/1000，装备三把测试法杖；第 4-5波均为 1000 怪，依次为「无池无 Worker / 仅池（关 Worker）」，用于同窗消融。第3 波含首次千怪冷启动，不纳入与第 4-5 波的公平对比。

表7-2 测试环境配置

项目	配置
操作系统	Windows 10/11（64 位）
运行方式	LayaAir IDE 内置浏览器预览（Chromium 内核）
视口 / 设计分辨率	1920×1080 预览；设计稿 1334×750 ，showall 缩放
引擎与语言	LayaAir 3.x, TypeScript
测试关卡	选关 Level3 ：5 波 × 10 s，双方无敌
统计工具	TestLevelFpsTracker（控制台日志）
工程版本	Git d0f08f5（2026-05-21 20:24:57 +0800，build.ps1 生成）

7.4.2 动态图集对比（前三波）

表 7-3 、7-4 记录接图集前后两轮采样，只看前三波。第 3 波 1000 怪：FPS 从13.32 到 15.10（约 +13.4%），弹幕多时合批有用。

表7-3 测试关卡分波 FPS（接入动态图集，前三波）

波次	同屏怪物 单发子弹		平均 FPS	采样帧数
1/3	10	10	56.74	569
2/3	100	100	40.72	408
3/3	1000	1000	15.10	152
前三波汇总	—		37.48	1129

表7-4 动态图集接入前后 FPS 对比（前三波）

波次（怪物数）	对照 FPS	动态图集 FPS	变化
1（10）	58.68	56.74	__1.94
2（100）	46.30	40.72	__5.58
3（1000）	13.32	15.10	+1.78
前三波汇总	39.46	37.48	__1.98

7.4.3 五波完整帧率

表 7-5 是同一次跑完的五波，已开图集和 DrawCall 合批。共 1733 帧、约 50.3 s，整场平均 FPS 约 34.48。

7.4.4 对象池消融（第4-5波）

结果解读

1. 规模曲线（第 1-3 波）。怪物 10→ 100→ 1000 时 FPS 由 58.62 降至 15.14，符合实体与弹幕压力上升规律。

表7-5 Level13 五波测试帧率（单次运行）

波次	怪物数	池	Worker	平均 FPS	P95 (ms)
1	10	开	开	58.62	—
2	100	开	开	46.84	—
3	1000	开	开	15.14	—
4	1000	关	关	26.16	76.50
5	1000	开	关	25.98	48.60
前五波汇总	—	—	—	34.48	—

表7-6 性能对比实验（对象池消融，1000 怪 / 10 s）

配置	平均 FPS	P95 (ms)	对应波次
基线（无池、无 Worker）	26.16	76.50	第 4 波
仅对象池（关 Worker）	25.98	48.60	第 5 波

参考：第 3 波（池 +Worker 全开，首次千怪）平均 FPS 15.14，不宜与上表直接对比。

2. 平均 FPS：池化对均值影响有限。第 4、5 波均值接近（26.16 对 25.98），预热后瓶颈主要在子弹碰撞与绘制。
3. P95。开池后 P95 从 76.50 ms 到 48.60 ms（约降 36.5%），和第 6 章对 GC/实例化的判断一致。
4. 第 3 波低于第 4 波基线。第 3 波含资源预热与首次千怪开销；对象池消融结论以第 4-5 波为准。

7.5 系统运行截图

截图源文件置于 thesis/Png/，编译前由 build.ps1 同步至 thesis/figures/。

图 7-1-7-6 覆盖元游戏与局内闭环。

7.6 测试结论

功能：Must 用 T-F-01-10 和 verify.ts 覆盖；Should 的跑图、装配、升级、池/Worker 也测过；Could 的法杖三槽 UI 和部分 Effect 特效没做完，和第 8 章一致。性能：图集让千怪波 FPS 高约 13.4%；五波整场均值

约 34.48；同窗消融里池把 P95拉低约 36.5%。

遗留：配置未同步至 bin/ 时图标可能空白（已用脚本与 isGameConfigReady 缓解）。已知缺陷：部分 Effect 仅逻辑生效、无独立 VFX，已在答辩范围中如实标注。



图7-1 游戏开始界面 (StartPanel)



图7-2 三大关 DAG 跑图界面 (RunMapPanel)



图7-3 局内战斗场面（同屏多怪与弹幕）



图7-4 Tab 暂停下的技能/Effect 装配面板



图7-5 生存胜利后的三选一奖励面板



图7-6 玩家死亡后的重启面板

7.7 本章小结

本章给出测试策略、单元验证、功能用例、环境与五波性能数据及解读；对象池价值已由第 4-5 波实测支持。图 7-1-7-6 为系统运行截图。

第八章 总结与展望

8.1 工作总结

本文围绕 Survivor And Fight，在 LayaAir 3.x 与 TypeScript 上完成 Roguelite生存战斗的分析、设计、实现与测试，对应关系如下：

- (1) 需求与选型（第 1-2 章）：梳理引擎架构、ECS、人群仿真与 H5 性能文献；确定 LayaAir、轻量 ECS、MVC UI、配表战斗的路线。
- (2) 总体设计（第 4 章）：给出分层结构、ECS/MVC/Worker 方案，以及时序、可靠性、容错和扩展方式。

(3) 实现（第 5 章）：以 SimpleEcsDemo 为根，实现技能链、子弹、怪物 AI、跑图、Tab 装配和主循环。

(4) 性能与测试（第 6-7 章）：千怪场景池化明显压低 P95；图集抬高重度弹幕波FPS；用例和脚本验证核心契约。

写作中参考 SyDRA[18]、Gregory 分层[7]和 Unity 人群实践[5]，把模块化、重算分离和 V&V 用到 H5 课设原型上。系统能演示菜单 → 跑图/选关 → 战斗 → 升级装配 → 死亡重启，达到毕设预期。

8.2 不足与展望

1. 内容与表现：法杖三槽 UI、部分 Effect 特效和音效仍缺；

2. 架构：ECS 可改 Archetype/Chunk 提同屏上限[7]；碰撞可加空间哈希；

3. 测试：补充端到端 UI 自动化与更长回归周期[18]；

4. 产品化：局外存档、元进度、帧率与同屏怪上限设置；

5. 发布：补美术与适龄说明后静态托管。

8.3 社会、健康与文化影响

战斗内容为虚构，怪物和血条偏抽象，避免写实血腥。原型无付费、无诱导分享、不采敏感数据。若上网发布，需遵守网游和未成年人保护规定，标适龄和时长；长时间玩注意眼疲劳，Tab 暂停和跑图休息节点用来调节节奏[18]。

8.4 绿色节能与兼容性

池化、暂停和 Worker 降低平均 CPU，对续航和发热有帮助[7]。配表双通道和Worker 回退提高浏览器兼容性；2D 对 GPU 压力小于大型 3D。后续可在设置里加帧率上限和同屏怪上限，做低功耗档。

8.5 本章小结

Survivor And Fight 说明在 H5 上用轻量 ECS、配表和可关断的性能手段做Roguelite 原型是可行的。论文叙述与仓库、测试表一致；后续可加强自动化测试和表现层，从答辩 Demo 往可发布小游戏再走一步。

致 谢

论文收尾之际，感谢指导教师在选题、架构和写作上的指点。

感谢计算机学院各位任课教师四年来的培养；感谢同学在项目调试与试玩中提供的帮助；感谢家人在毕业设计期间的理解与支持。

由于本人水平有限，文中疏漏之处恳请各位老师批评指正。

参考文献

[1] BAABAD A, ZULZALIL H B, HASSAN S, et al. Software architecture degradation in open source software: A systematic literature review[J]. IEEE Access, 2020, 8: 173681-173709.

[2] GREGORY J. Game engine architecture[M]. 3rd ed. Boca Raton: CRC Press, 2018.

[3] MUNRO J, BOLDYREFF C, CAPILUPPI A. Architectural studies of games engines -- the quake series[C]//Proceedings of the 2009 International IEEE Consumer Electronics Society's Games Innovations Conference. [S.l.]: IEEE, 2009: 246-255.

[4] AGRAHARI V, CHIMALAKONDA S. What's inside unreal engine? - A curious gaze![C]//Proceedings of

- the 14th Innovations in Software Engineering Conference. [S.l.]: ACM, 2021: 1–5.
- [5] GOSLIN M, MINE M R. The Panda3D graphics engine[J]. Computer, 2004, 37(10): 112–114.
- [6] ULLMANN G C, GUÉHÉNEUC Y G, PETRILLO F, et al. SyDRA: An approach to understand game engine architecture[J]. Entertainment Computing, 2025, 52: 100832.
- [7] ULLMANN G C, GUÉHÉNEUC Y G, PETRILLO F, et al. Visualising game engine subsystem coupling patterns[C]//Lecture Notes in Computer Science: Vol. 14455 Entertainment Computing -- ICEC 2023. [S.l.]: Springer, 2023: 263–274.
- [8] Unity Technologies. Unity manual[EB/OL]. (2016) [2026-05-17]. <https://docs.unity3d.com/Manual/index.html>.
- [9] BRIAND L C, BUNSE C, DALY J W. A controlled experiment for evaluating quality guidelines on the maintainability of object-oriented designs[J]. IEEE Transactions on Software Engineering, 2001, 27(6): 513–530.
- [10] REYNOLDS C W. Flocks, herds, and schools: A distributed behavioral model[C]//Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques. [S.l.]: ACM, 1987: 25–34.
- [11] THALMANN D, MUSSE S R. Crowd simulation[M]. London: Springer, 2007.
- [12] CANTRELL W A, PETTY M D, KNIGHT S L, et al. Physics-based modeling of crowd evacuation in the Unity game engine[J]. International Journal of Modeling, Simulation, and Scientific Computing, 2018, 9(4): 1850029.
- [13] BOTT M, MUSSE S R, DE OLIVEIRA L P, et al. Implementing a physics based model of crowd movement using unreal development kit[J]. Journal of Gaming & Virtual Worlds, 2014, 6(3): 275–296.
- [14] HELBING D, FARKAS I, VICSEK T. Simulating dynamical features of escape panic[J]. Nature, 2000, 407: 487–490.
- [15] MCKENZIE F D, PETTY M D, KRUSZEWSKI P A, et al. Integrating crowd-behavior modeling into military simulation using game technology[J]. Simulation & Gaming, 2008, 39(1): 10–38.
- [16] OBERKAMPF W L, ROY C J. Verification and validation in scientific computing[M]. Cambridge, UK: Cambridge University Press, 2010.
- [17] HEIJSTEK W, KÜHNE T, CHAUDRON M R V. Experimental analysis of textual and graphical representations for software architecture design[C]//Proceedings of the 2011 International Symposium on Empirical Software Engineering and Measurement. [S.l.]: IEEE, 2011: 167–176.
- [18] PETTY M D. Verification, validation, and accreditation[M]//Modeling and Simulation Fundamentals. [S.l.]: John Wiley & Sons, 2010: 325–372.

附录

附录A 名词术语及缩略词

术语/缩略语	含义
--------	----

ECS	Entity Component System，实体组件系统
Roguelite	轻 Roguelike，含元进度与单局随机
DAG	Directed Acyclic Graph，有向无环图
H5	HTML5 网页游戏
UI2	LayaAir 新一代 UI 组件（GBox/GButton 等）
Worker	Web Worker，浏览器后台线程
MoSCoW	Must/Should/Could/Won’ t 需求优先级方法
V&V	Verification and Validation，验证与确认
P95	第 95 百分位帧时间，反映卡顿尖峰

附录B 核心配表字段示例（节选）

Character 表字段含 id、prefabPath、bodyPrefabPath、hp、maxHp、roleType。玩家和怪物同表，靠 roleType 选预制体和默认值。

Bullet 表：id、prefabPath、duration、speed、damage、penetration。子弹系统按 id 查表生成运动参数与伤害。

Skill 表：id、cooldown、effectSlotCount、图标路径等。一个技能可关联多行SkillEffect。

SkillEffect 表：id、effect、damage、params、iconPath。effect 取值包括bullet、modifier_split、modifier_chain、modifier_pierce、direct_damage。

完整 JSON 在 docs/config/。运行前执行 tools/sync-config-to-bin.ps1，把表拷到 bin/config/。

附录C 需求与测试用例映射（节选）

表 C-1 给出需求标识与测试用例、验证方式的对应关系，便于答辩时说明“需求—测试”可追溯性。

表C-1 需求—测试映射（节选）

需求 ID	需求摘要	验证方式
R-ECS-01	实体销毁时清理全部组件	ecsCorePhase1.verify.ts
R-ECS-02	系统按分组与优先级顺序执行	ecsCorePhase1.verify.ts
R-SKILL-01	三装备栏独立冷却与自动施法	T-F-04，观察多技能轮替
R-SKILL-02	Tab 打开面板暂停战斗	T-F-05 ，GameSession.paused
R-SKILL-03	拖拽装配写回战斗实体	T-F-06，装配同步用例
R-BULLET-01	穿透/分裂/连锁 按配表生效	T-F-04，高密度战斗观察
R-MON-01	波次刷怪与追逐分离	T-F-03 ，Worker 一致性对比
R-META-	主菜单进入战斗与计	T-F-09

01	时	
R-MAP-01	跑图 DAG Boss 可达	100 次随机种子断言
R-PERF-01	对象池降低帧时间尖峰	表 7-6，第 4-5 波
R-PERF-02	动态图集提升千怪FPS	表 7-4
R-UI-01	升级奖励三选一	T-F-07
R-UI-02	死亡重启面板	T-F-08

附录D 缺陷记录（项目实测）

表D-1 缺陷记录表

ID	描述	严重性	状态
B-01	配表未同步至 bin/config 导致技能图标空白	中	已修复（同步脚本）
B-02	Worker 脚本路径错误时回退主线程	低	已修复
B-03	部分 Effect 仅 UI 展示无独立 VFX	低	已知，论文已说明
B-04	法杖三槽运行时与专用 UI 未实现	中	规划项，第 8 章
B-05	千怪首次波次 FPS 低于预热后基线	低	已分析，见第 7 章

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告结合AI大模型自动生成，结果仅供参考。
- 报告支持中、英文内容检测。



唯一官网：<https://vpcs.fanyu.com> | 客服邮箱：vpcs@fanyu.com | 客服热线：400-607-5550 | 客服QQ：4006075550